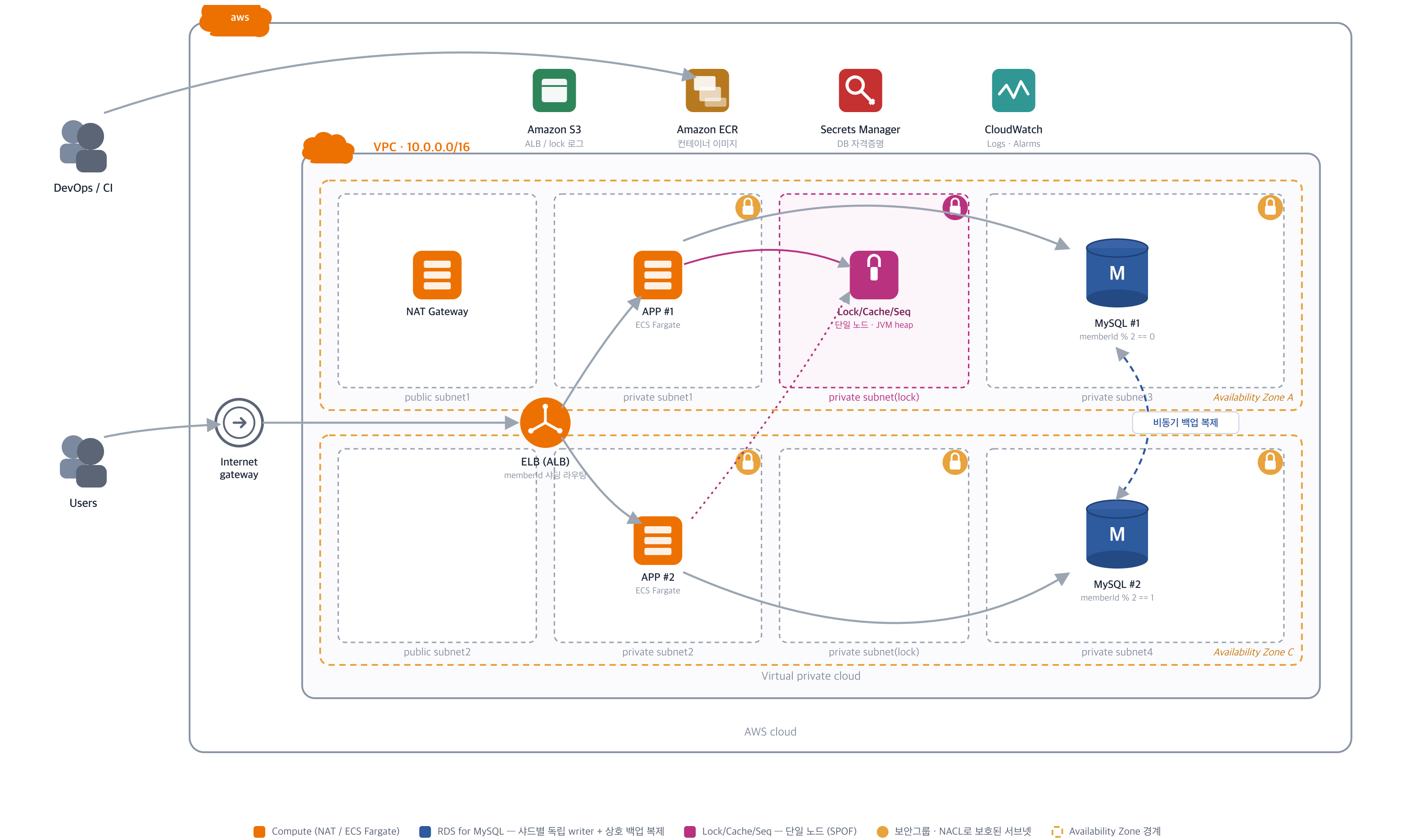


Point API를 위한 AWS 배포 아키텍처 — 서버당 독립 DB(샤딩)



요청 흐름

1. Users → Internet Gateway → ELB(ALB) — memberId % 2 샤드 키로 APP#1(짝수) / APP#2(홀수)에 고정 라우팅
2. ELB → APP#1 / APP#2(ECS Fargate, private subnet) — health check GET /actuator/health
3. 적립/사용 API 호출 시 @DistributedLock AOP는 두 APP이 공유하는 Lock/Cache/Seq 노드(전용 서브넷, 단일)에 락 획득 요청 — 로직은 기존 MemoryDistributedLockExecutor 그대로
4. 락 획득 후 각 APP은 자기 샤드 DB에만 지갑·원장을 갱신한다 — MySQL#1 ↔ APP#1, MySQL#2 ↔ APP#2, 쓰기는 서로 독립
5. 두 MySQL은 비동기 백업 복제로 상대 인스턴스에 데이터를 미러링한다 — 한쪽 AZ 전체 장애 시 다른 쪽에서 복구 가능
6. DevOps/CI → ECR에 이미지 푸시 → ECS 롤링 배포로 APP#1/#2 갱신

모듈 → AWS 배치 매핑

현재 구현 (INFRASTRUCTURE)	AWS 배치
H2 file DB (jdbc:h2:file:)	RDS for MySQL × 2 — memberId 모듈러 샤딩, 쓰기는 독립 + 비동기 백업 복제로 연동
MemoryDistributedLockExecutor	로직 동일, 전용 노드(desiredCount=1)로 분리 + 네트워크 클라이언트 신설
MemoryCacheExecutor / MemoryDistributedKeyGenerator	같은 Lock/Cache/Seq 노드 안에서 함께 서빙

트레이드오프 — Lock/Cache/Seq 노드가 곧 SPOF다. 재시작 시 락·캐시·채번 상태가 초기화되고 desiredCount는 반드시 1로 고정해야 한다.

샤딩 참고 — memberId % N 방식은 서버를 2대→3대로 늘리는 순간 매핑이 전부 바뀌어 재배포가 필요하다. 또한 특정 memberId 요청이 항상 같은 APP으로만 가므로, 사실 락도 서버 로컬 in-memory로 충분해질 여지가 있다 — 이 다이어그램은 Lock/Cache/Seq 공유 모델을 그대로 유지한 상태에서 DB만 분리한 절충안이다.